



Universidad Autónoma de Nayarit



Unidad académica de economía

Carrera:

Sistemas computacionales

Materia:

Programación del lado del cliente

Tema:

Reto IA 6

Maestro:

Eligardo Cruz Sanches

Alumna:

Erika Alejandra Orozco Vázquez

ADR 001: Diseño Inicial del Cliente HTTP (MVP Fase 2.1)

Estado: Aceptado (como versión MVP inicial), con áreas identificadas para futura refactorización.

Fecha: 18 de mayo de 2026

1. Contexto

Se requiere construir un cliente HTTP en JavaScript para interactuar con una API que maneja productos. El proyecto se encuentra en una etapa temprana (MVP / Primer Avance), por lo que se prioriza la simplicidad, la rapidez de desarrollo y el uso de los conocimientos técnicos actuales del desarrollador. No hay requerimientos inmediatos para múltiples entornos concurrentes, esquemas de datos masivos o manejo avanzado de redes inestables.

2. Decisiones Tomadas

- Estructura del Cliente (Funciones vs Clases):** Se decide utilizar funciones independientes a nivel de módulo (listarProductos, crearProducto) apoyadas por un estado global (BASE_URL) en lugar de instanciar clases de cliente.
 - Justificación:* Simplicidad (KISS) y facilidad para desglosar la lógica en apartados específicos.
- Validación de Datos:** Se decide implementar una validación manual estructurada iterando con if/else y typeof (ej. validarProductoJS), en lugar de integrar librerías de terceros (como Zod o Pydantic/Joi).
 - Justificación:* Mayor control manual sobre las excepciones y mantenimiento de un cliente con "cero dependencias" (zero-dependency).
- Gestión de Timeouts:** Se decide aplicar un *timeout* fijo estándar (8-10 segundos) a nivel global en el wrapper de la API.
 - Justificación:* Aplicación del principio YAGNI (No lo vas a necesitar todavía); el abanico actual de peticiones no justifica la complejidad de una configuración dinámica por ruta.
- Resiliencia (Reintentos automáticos):** Se decide fallar inmediatamente (throw err) sin intentar re-ejecutar las peticiones fallidas por problemas de red transitorios.
 - Justificación:* Curva de aprendizaje actual. Es preferible un fallo directo y predecible que implementar un mecanismo complejo mal manejado que pueda derivar en problemas graves de **idempotencia** (ej. crear productos duplicados).
- Manejo de Errores:** Se decide lanzar errores nativos de JavaScript (new Error ()) y gestionar la lógica de captura (catch) buscando coincidencias de texto en el mensaje (ej. err.message.includes('404')).
 - Justificación:* Pragmática de desarrollo basada en las herramientas y patrones actualmente dominados por el desarrollador.

3. Consecuencias (Impacto)

Fortalezas (Impacto Positivo)

- **Baja fricción:** El código actual es fácil de leer, no está sobre-ingeniado y cumple su propósito principal eficientemente.
- **Ligereza:** Al no depender de librerías externas para la validación, el cliente es altamente portable.
- **Observabilidad funcional:** Se incluyó un robusto sistema de logs estructurados (apiFetchConLogging) y sanitización de *headers*, lo cual facilita la depuración de las decisiones tomadas.

Riesgos y Deuda Técnica a Mitigar (Impacto Negativo a Futuro)

- **Escalabilidad del Estado:** Si la app necesita conectarse a dos APIs simultáneamente o manejar diferentes tokens de usuario en paralelo, el estado global (BASE_URL) será un bloqueador. Se requerirá migrar a un patrón de fábrica o clases.
- **Mantenibilidad de las Validaciones:** A medida que la entidad "Producto" adquiera más campos y anidaciones, las validaciones imperativas se volverán muy frágiles ("código espagueti").
- **Experiencia de Usuario (UX) frágil:** Ante micro-cortes de red, el sistema parecerá "congelado" hasta el timeout y no será capaz de auto-recuperarse, lo que puede frustrar al usuario final.
- **Fragilidad en Errores:** Depender de cadenas de texto específicas (includes('404')) para tomar decisiones lógicas hace que el sistema sea susceptible a romperse silenciosamente si el backend cambia los mensajes de error. Se deberá migrar a lectura de Status Codes HTTP o Clases de Error Extendidas.

Conclusión:

Después de este análisis me di cuenta de algunos puntos en los que tengo que poner enfoque en el desarrollo para mejorar el cliente uno de ellos es lo de las alertas de los errores y la manera de validar los datos , probablemente surgan cambios en esas secciones.